State-of-the-Art Quantitative Finance Algorithms: A Comprehensive Review of Stochastic, Machine Learning, and Hybrid Frameworks (2024-2025)

1. Introduction: The Convergence of Paradigms

The discipline of quantitative finance is currently undergoing a structural metamorphosis of a magnitude not seen since the introduction of the Black-Scholes-Merton framework in the 1970s. For decades, the field was bifurcated into two distinct magisteria: "Q-quant" (or derivatives pricing), which operated under risk-neutral measures using rigorous stochastic calculus to ensure arbitrage-free pricing; and "P-quant" (or risk and portfolio management), which operated under physical measures using statistical and econometric techniques to forecast real-world asset dynamics.

In the period spanning 2024 to 2025, this dichotomy has largely collapsed. The catalyst for this unification is the emergence of **Hybrid Neuro-Stochastic Architectures**—algorithms that fuse the universal approximation capabilities of deep neural networks with the structural guarantees of stochastic differential equations (SDEs) and partial differential equations (PDEs). We are no longer witnessing a competition between "black box" machine learning (ML) and "white box" traditional models; rather, we are seeing the rise of "grey box" systems where neural networks learn the solution operators of complex physical equations, and where stochastic calculus provides the inductive bias necessary for ML models to generalize in non-stationary markets.

This report provides an exhaustive, expert-level analysis of the state-of-the-art (SOTA) algorithms defining this new era. The analysis is organized into three primary domains:

1. **Traditional Stochastic Algorithms:** Focusing on the industrialization of Rough Volatility and the integration of default risk into pricing kernels.
2. **Machine Learning Algorithms:** Detailing the shift from supervised prediction to Agentic Workflows, Multi-Agent Reinforcement Learning (MARL), and Generative AI for Limit Order Books.
3. **Hybrid and Physics-Informed Algorithms:** Exploring the frontier of Neural SDEs, Finite Dimensional Matching (FDM), and Deep Operator Networks (DeepONets) for real-time calibration.

The following sections rigorously detail the mathematical formulations, implementation challenges, and benchmark performance of these algorithms, drawing on the most recent

literature and empirical studies.

---

2. Traditional Quantitative Algorithms: The Rough Volatility Revolution and Beyond

While machine learning dominates headlines, the bedrock of quantitative finance remains stochastic calculus. The most significant advancement in this domain over the last two years has been the maturation of **Rough Volatility** models from academic curiosities to industrial standards, driven by their superior ability to fit the implied volatility surface and capture the time-series properties of realized variance.

2.1 The Phenomenology of Roughness

The "Rough Volatility" paradigm challenges the classical assumption, embedded in models like Heston (1993) or Hull-White (1987), that the volatility process is a continuous semimartingale driven by standard Brownian motion. Under standard Brownian motion, the trajectories of volatility are continuous but "smooth" in a specific probabilistic sense: they exhibit Hölder regularity with an exponent $\alpha \approx 0.5$.

However, extensive empirical analysis of high-frequency data reveals that the log-volatility of asset prices behaves differently. It exhibits a scaling property consistent with **Fractional Brownian Motion (fBm)** where the Hurst parameter H is significantly less than 0.5, typically $H \approx 0.1$. This implies that volatility paths are much "rougher" or more jagged than standard diffusion processes allow. This roughness is not merely noise; it is a structural feature that explains the "explosive" behavior of the at-the-money (ATM) volatility skew as time-to-maturity approaches zero.

2.1.1 Mathematical Formulation of Fractional Brownian Motion

To understand the SOTA traditional models, one must first define the driving noise. A Fractional Brownian Motion $(W^H_t)_{t \ge 0}$ is a continuous centered Gaussian process with covariance function:

$$\mathbb{E} = \frac{1}{2} \left(|t|^{2H} + |s|^{2H} - |t-s|^{2H} \right)$$
For the case where $H < 0.5$ (the rough regime), the increments of the process are negatively correlated (anti-persistent). This anti-persistence is crucial for modeling the mean-reverting nature of volatility at multiple timescales. Unlike standard Brownian motion ($H=0.5$), fBm is neither a semimartingale (preventing the direct use of Itô calculus) nor a Markov process, which historically made pricing derivatives on such assets computationally intractable.

2.2 The Rough Bergomi (rBergomi) Model

The **Rough Bergomi** model has emerged as the benchmark for pricing equity derivatives under this new paradigm. It modifies the classical Bergomi variance curve model by replacing the exponential kernel with a power-law kernel derived from the properties of fBm.

The forward variance $\xi_t(u) = \mathbb{E}[v_u | \mathcal{F}_t]$ in the rBergomi framework is modeled as:

$$\xi_t(u) = \xi_0(u) \exp \left(\eta \int_0^t \frac{dW_s}{(t - s)^\gamma} - \frac{1}{2} \eta^2 \int_0^t \frac{ds}{(t - s)^{2\gamma}} \right)$$
where:

- $\eta > 0$ is the volatility-of-volatility parameter.
- $\gamma = \frac{1}{2} - H$ represents the degree of roughness. For $H=0.1$, $\gamma=0.4$, implying a singular kernel that assigns high weight to recent events.
- W_s is a standard Brownian motion, but the convolution with the singular kernel $(t-s)^{-\gamma}$ induces the fractional behavior in the resulting variance process.

2.2.1 Implication for the Volatility Skew

The power of the rBergomi model lies in its ability to reproduce the power-law decay of the ATM skew term structure observed in market data. In classical stochastic volatility models, the skew $\psi(\tau)$ behaves as $\psi(\tau) \sim \text{const}$ for short maturities $\tau \to 0$. In contrast, market data consistently shows $\psi(\tau) \sim \tau^{-H}$.

The rBergomi model naturally generates this behavior:

$$\psi(\tau) \approx \frac{\eta}{\tau^\gamma} = \frac{\eta}{\tau^{0.5-H}}$$
This alignment with empirical "stylized facts" makes rBergomi superior for calibration, particularly for short-dated options where classical models fail to capture the steepness of the smile.

2.3 The Rough Heston Model and Characteristic Functions

While rBergomi is powerful, it lacks a closed-form characteristic function, necessitating Monte Carlo simulation for pricing. The **Rough Heston** model addresses this by providing a semi-analytical framework using affine structures. It is defined by the SDEs:

$$dS_t = S_t \sqrt{V_t} dW_t$$

$$V_t = V_0 + \frac{\lambda}{\Gamma(\alpha)} \int_0^t (t-s)^{\alpha-1} (\theta - V_s) ds + \frac{\lambda \nu}{\Gamma(\alpha)} \int_0^t (t-s)^{\alpha-1} \sqrt{V_s} dB_s$$

Here, the variance process V_t is a fractional integrator of a mean-reverting square-root diffusion. The characteristic function of the log-price can be expressed in terms of the solution to a **Fractional Riccati Equation**:

$$D^\alpha h(u, s) = -\frac{1}{2}u(u+i) + \lambda(i\rho\nu u - 1)h(u, s) + \frac{(\lambda\nu)^2}{2}h^2(u, s)$$
where D^α is the fractional derivative operator. Solving this Fractional Riccati equation numerically allows for much faster pricing than pure simulation, bridging the gap between the realism of roughness and the speed requirements of trading desks.

2.4 Vulnerable Options and Default Risk Integration

Moving beyond pure volatility modeling, recent advancements in 2024-2025 have integrated **Counterparty Credit Risk** directly into option pricing kernels. This is particularly relevant for Over-the-Counter (OTC) derivatives where the option writer's default is a material risk.

New frameworks extend the Heston model to price "vulnerable options" by modeling the correlation between the underlying asset, the stochastic volatility, and the *credit quality* of the option issuer. A key innovation is the introduction of a **stochastic long-term mean** for the volatility process.

In standard Heston, the long-term mean θ is constant. In the generalized vulnerable pricing model, θ itself follows a stochastic process:

$$dV_t = \kappa (\theta_t - V_t) dt + \sigma_V \sqrt{V_t} dW_t^V$$

$$d\theta_t = \kappa_\theta (\bar{\theta} - \theta_t) dt + \sigma_\theta \sqrt{\theta_t} dW_t^\theta$$
This hierarchical structure (volatility of volatility of mean) allows the model to capture term structures of implied volatility that twist and turn over time, a feature often observed during credit crises. The option value is then derived not just as a discounted expectation of the payoff, but as an expectation conditioned on the survival of the issuer:

$$\text{Price} = \mathbb{E}^\mathbb{Q} \left[$$
where λ_s is the stochastic default intensity, often correlated with V_t (since high volatility often implies higher credit risk). Analytical solutions for these coupled systems rely on asymptotic expansions or affine transform techniques, representing the frontier of "solvable" traditional models.

---

3. Machine Learning Algorithms: The "New Quant" Era

If traditional algorithms are defined by the refinement of stochastic assumptions, Machine Learning (ML) algorithms in finance are defined by their ability to learn complex, non-linear mappings from vast, heterogeneous datasets. The period 2024-2025 has seen a definitive shift from simple supervised learning (e.g., predicting next-day returns with LSTMs) to **Agentic** and **Generative** paradigms.

3.1 Deep Reinforcement Learning (DRL) in Multi-Agent Markets

Reinforcement Learning (RL) has matured significantly. The SOTA has moved from single-agent optimizations (which fail in competitive markets) to **Multi-Agent Reinforcement Learning (MARL)** systems that can account for market impact and the strategic behavior of other participants.

3.1.1 Asynchronous Advantage Actor-Critic (A3C)

In high-frequency domains like Forex (FX) trading, the **Asynchronous Advantage Actor-Critic (A3C)** algorithm has become a dominant architecture. Unlike Deep Q-Networks (DQN) which rely on a replay buffer, A3C utilizes multiple worker agents interacting with parallel environments to update a global policy.

Mathematical Formulation:
The global network maintains a policy $\pi(a_t|s_t; \theta)$ and a value function $V(s_t; \theta_v)$. Each worker i interacts with its own instance of the market (e.g., different currency pairs or time periods) and computes gradients to update the global parameters.
The loss function minimized by the A3C algorithm is a composite of the policy gradient loss, the value function loss, and an entropy regularization term:

$$\mathcal{L} = \mathcal{L}_\pi + \alpha \mathcal{L}_v - \beta \mathcal{H}$$
Where:

- **Policy Loss ($\mathcal{L}_\pi$):** $\nabla_\theta \log \pi(a_t|s_t; \theta) A(s_t, a_t)$, where $A(s_t, a_t) = R_t - V(s_t)$ is the advantage function. This term encourages actions that lead to higher-than-expected returns.
- **Value Loss ($\mathcal{L}_v$):** $(R_t - V(s_t; \theta_v))^2$. This term trains the critic to accurately estimate the expected return of a state.
- **Entropy ($\mathcal{H}$):** $-\sum \pi(a|s) \log \pi(a|s)$. This term prevents premature convergence to deterministic policies, ensuring exploration.

Recent benchmarks [1] indicate that A3C architectures with "Lock" mechanisms (synchronization barriers) outperform Proximal Policy Optimization (PPO) in multi-currency scenarios. The parallel nature of A3C allows it to decorrelate the training data, stabilizing

learning in the highly non-stationary FX market.

3.1.2 PipelineRL: Solving the Data Freshness Problem

A major bottleneck in applying RL to large-scale financial models (such as FinLLMs) is the computational cost of generating "rollouts" (simulated trading sequences). In standard RL, the GPU is often idle while waiting for the CPU to generate data, or vice versa.

PipelineRL, introduced in late 2024/early 2025, addresses this with a novel **In-Flight Weight Update** mechanism.

- **Traditional:** Generate Data $\to$ Stop Generation $\to$ Update Model $\to$ Resume Generation.
- **PipelineRL:** The inference engine (Actor) receives updated model weights from the Learner *continuously*, even while generating a sequence.

Mathematically, this minimizes the **Policy Lag** (π_{old} vs π_{new}). If the data is generated by an old policy π_{old} but used to train π_{new}, the "off-policyness" introduces bias. PipelineRL ensures that the Kullback-Leibler (KL) divergence $D_{KL}(\pi_{new} |

| \pi_{old})$ remains minimal without stalling the hardware. Benchmarks using 128 H100 GPUs show a 2x speedup in training throughput compared to standard baselines.[2]

3.2 Large Language Models (LLMs) and Agentic Workflows

The application of LLMs in finance has evolved from passive sentiment analysis to active **Agentic Workflows**. Models are no longer just classifiers; they are autonomous agents capable of reasoning, tool use, and decision-making.

3.2.1 The FinMem and FinAgent Architectures

Systems like **FinMem** represent the SOTA in this domain. They utilize a modular cognitive architecture:

1. **Perception Module:** Ingests multimodal data (earnings call transcripts, price ticks, news headlines).
2. **Memory Module:** A vector database (e.g., using RAG - Retrieval Augmented Generation) that stores historical context and successful reasoning patterns. This allows the agent to "remember" how similar market conditions played out in the past.
3. **Reasoning Module:** Uses Chain-of-Thought (CoT) prompting to decompose complex trading decisions into intermediate steps (e.g., "Analyze inflation data" $\to$ "Check impact on bond yields" $\to$ "Adjust equity position").
4. **Action Module:** Connects to execution APIs to place trades.

QuantBench results [3] demonstrate that these agentic systems outperform traditional

single-step deep learning models (like LSTM or XGBoost) in metrics of "Financial Logic" and "Temporal Stability." However, they can still be prone to hallucinations, necessitating "Reflexion" loops where the agent critiques its own plan before execution.

3.3 Generative Modeling for Limit Order Books (LOB)

Limit Order Books (LOBs) represent the microscopic state of supply and demand. Modeling the full distribution of LOB updates is critical for training RL agents (as realistic simulators) and for identifying liquidity anomalies.

3.3.1 State Space Models (SSMs) and S5 Layers

The current SOTA for LOB generation is not the Transformer (which scales quadratically with sequence length) but **Structured State Space Models (SSMs)**, specifically the **S5** (Simplified State Space Layers) architecture.

The LOB is treated as a sequence of updates u_t. The S5 layer models the continuous-time evolution of the hidden state $h(t)$ via a linear ODE, which is then discretized:

$$h'(t) = \mathbf{A} h(t) + \mathbf{B} x(t)$$

$$y(t) = \mathbf{C} h(t) + \mathbf{D} x(t)$$

By diagonalizing the state matrix $\mathbf{A}$, the S5 layer can be computed efficiently as a parallel scan (associative convolution). This allows the model to capture extremely long-range dependencies in order flow (e.g., an institutional buy order executed over hours) that Transformers struggle to retain.

Benchmarks on **LOB-Bench** [4] show that S5-based generators achieve the lowest Wasserstein distance between real and synthetic LOB distributions, significantly outperforming GANs and diffusion models in capturing the "burstiness" and cross-correlation of market depth.

---

4. Hybrid Neuro-Stochastic Algorithms: The Convergence

The most profound theoretical developments are occurring at the intersection of the two previous domains. **Hybrid Algorithms** fuse the interpretability and constraints of stochastic calculus with the flexibility of neural networks. These are not merely ensembles; they are deeply integrated mathematical frameworks.

4.1 Neural Stochastic Differential Equations (Neural SDEs)

A Neural SDE parameterizes the drift and diffusion coefficients of a standard SDE with neural

networks. This allows the model to learn the *dynamics* of the system rather than just the output distribution.

Formulation:

$$dZ_t = \mu_\theta(t, Z_t) dt + \sigma_\theta(t, Z_t) dW_t$$

$$X_t = \text{Readout}(Z_t)$$
Here, μ_θ and σ_θ are neural networks with weights θ. This formulation ensures that the generated paths X_t are continuous and can be subjected to standard financial calculus (e.g., computing Greeks via Malliavin calculus).

4.1.1 Training via Finite Dimensional Matching (FDM)

A critical breakthrough in 2025 regarding Neural SDEs is the training methodology. Previously, training required matching the "signature" of the paths using Signature Kernels, which involved solving a linear PDE. This was computationally expensive, scaling quadratically $O(D^2)$ with the number of time discretization steps D.

The new SOTA method is **Finite Dimensional Matching (FDM)**. This approach leverages the **Markov property** inherent in the SDE formulation. If the process is Markovian, the entire law of the process is determined by its marginal distributions and transition probabilities.

The FDM Scoring Rule:
Instead of comparing full paths, FDM minimizes a Strictly Proper Scoring Rule S between the finite-dimensional distributions of the model P_θ and the data P_{data}:

$$\mathcal{L}_{FDM}(\theta) = \sum_{i=1}^M \mathbb{E}_{x \sim P_{data}}$$
This reduces the complexity from $O(D^2)$ to $O(D)$, making it feasible to train Neural SDEs on long, high-frequency financial time series. The resulting models generate synthetic paths that are statistically indistinguishable from real market data but fully differentiable with respect to their parameters.[5]

4.2 Physics-Informed Neural Networks (PINNs): DeepSVM

While Neural SDEs are generative, **Physics-Informed Neural Networks (PINNs)** are used for calibration and solving pricing equations. The SOTA model here is **DeepSVM** (Physics-Informed Deep Operator Network for Stochastic Volatility Models).

Standard calibration involves finding parameters Θ (e.g., Heston parameters) that minimize the error between model prices $C_{model}(\Theta)$ and market prices C_{market}. This requires solving the pricing PDE inside an optimization loop—a slow process. DeepSVM replaces this with **Operator Learning**.

4.2.1 DeepONet Architecture and Operator Learning

DeepSVM learns the operator $\mathcal{G}$ that maps parameters to the solution function:

$$\mathcal{G}: \Theta \to u(S, \nu, t)$$
The architecture consists of two sub-networks:

1. **Branch Net:** Takes the parameters $\Theta = (\kappa, \theta, \sigma, \rho)$ as input and outputs a feature vector $[b_1, \dots, b_p]$.
2. **Trunk Net:** Takes the coordinates (S, ν, t) as input and outputs a basis vector $[t_1, \dots, t_p]$.

The approximate price is the dot product:

$$u(S, \nu, t; \Theta) \approx \sum_{k=1}^p b_k(\Theta) \cdot t_k(S, \nu, t)$$

4.2.2 The Physics-Informed Loss and Hard Constraints

The model is trained to minimize the residual of the Heston PDE. If $\mathcal{N}[u] = 0$ is the Heston PDE operator:

$$\mathcal{N}[u] = \frac{\partial u}{\partial t} + \frac{1}{2}S^2 v \frac{\partial^2 u}{\partial S^2} + \rho \sigma v S \frac{\partial^2 u}{\partial S \partial v} + \dots - r u$$
The loss function is $\mathcal{L} = ||\mathcal{N}[u_{pred}]||^2$.

A crucial innovation in DeepSVM is the **Hard-Constrained Ansatz**. Standard PINNs struggle to satisfy the initial condition (payoff) exactly. DeepSVM enforces it by construction:

$$u_{pred}(S, \nu, \tau) = \text{Payoff}(S) + \tau \cdot \text{Net}(S, \nu, \tau)$$
At $\tau=0$ (maturity), the second term vanishes, and the price is exactly the payoff. This guarantees that the model is arbitrage-free at maturity and accelerates training convergence. Once trained, DeepSVM calibrates to market data in milliseconds (a simple forward pass), compared to seconds or minutes for finite difference solvers.[7]

4.3 Signature-Based Methods

The final hybrid category utilizes **Path Signatures** from Rough Path Theory. The signature $S(X)$ of a path X is the infinite sequence of its iterated integrals:

$$S(X)_{s,t} = \left(1, \int_s^t dX_u, \int_s^t \int_s^u dX_v \otimes dX_u, \dots \right)$$
Signatures provide a universal basis for continuous functions on paths. In recent hybrid models, the volatility process itself is modeled as a linear function of the signature of the price

path (and potentially other factors):

$$\sigma_t^2 \approx \langle \ell, S(X)_{0,t}^{\le N} \rangle$$
This formulation transforms the highly non-linear, path-dependent volatility modeling problem into a **Linear Regression** problem. By regressing the observed market variance (e.g., from VIX or variance swaps) against the truncated signatures of the asset price history, quants can calibrate complex path-dependent volatility models instantaneously. This method is particularly powerful because the signature naturally encodes the order of events (e.g., "price dropped *then* volatility spiked" vs "volatility spiked *then* price dropped"), which simple correlation metrics miss.[8]

---

5. Comparative Analysis and Benchmarking

To rigorously evaluate these algorithms, the industry has moved towards standardized benchmark suites.

5.1 Benchmarking Frameworks

5.1.1 LOB-Bench

LOB-Bench [4] focuses on the fidelity of generative models for high-frequency data.

- **Key Metrics:**
  - **Distributional Distance:** Wasserstein-1 distance between the cumulative distribution functions (CDFs) of real and synthetic order attributes (price, volume, inter-arrival time).
  - **Discriminator Score:** The accuracy of a classifier trained to distinguish real from fake data. A score of 0.5 implies perfect indistinguishability.
  - **Stylized Facts:** Ability to reproduce the "volatility clustering" and "volume autocorrelation" of real markets.
- **Result:** S5-based models consistently outperform GANs and diffusion models, primarily because they do not suffer from mode collapse and better capture temporal dependencies.

5.1.2 QuantBench

QuantBench [9] evaluates LLMs and Agents.

- **Key Metrics:**
  - **Information Coefficient (IC):** Correlation between predicted scores and actual future returns.
  - **Sharpe/Sortino Ratios:** Risk-adjusted returns of the generated strategies.
  - **Bias Scores:** Quantifying "Look-ahead Bias" (using future data for training) and "Survivorship Bias" (ignoring delisted companies).

- **Result:** Agentic systems (e.g., FinAgent) dominate static models. However, they are computationally expensive and require careful prompt engineering to avoid "hallucinating" non-existent financial instruments.

5.2 Summary Comparison Table

Feature	Traditional (Heston, rBergomi)	Machine Learning (RL, LLM)	Hybrid (Neural SDE, DeepSVM)
Mathematical Core	Stochastic Calculus, PDEs	Gradient Descent, Transformers	Operator Learning, Rough Paths
Data Requirement	Low (calibrated to daily quotes)	High (requires tick/text history)	Medium (Physics priors reduce need)
Interpretability	High (Mean reversion κ, Vol-of-Vol η)	Low (Attention weights, neuron activations)	Medium (Params map to dynamics)
Computational Speed	Slow (Online PDE solving)	Fast Inference / Slow Training	**Fastest** (Amortized Inference)
Flexibility	Rigid (Model assumptions fixed)	High (Learns any pattern)	Constrained Flexibility (Arbitrage-free)
Best Use Case	Regulatory Reporting, Vanilla Pricing	High-Frequency Trading, Sentiment Analysis	Real-Time Risk, Exotic Calibration

---

6. Conclusion and Future Outlook

The trajectory of quantitative finance algorithms in 2025 is unmistakable: **Integration**. The most successful approaches are those that do not discard the theoretical achievements of the past 50 years but instead embed them into modern computational architectures.

Key Takeaways:

1. **Roughness is Fundamental:** Volatility is not a smooth diffusion; it is rough. Traditional models have adapted via fractional calculus (rBergomi), while hybrid models capture this

via Neural SDEs trained on high-frequency data.
2. **Physics-Informed Learning is the New Standard for Calibration:** Solving PDEs via finite differences is obsolete for real-time applications. Learning the solution operator (DeepSVM) allows for instant, arbitrage-free pricing across the entire parameter surface.
3. **Agents Replace Predictors:** In the ML domain, static prediction models are being replaced by dynamic Agents that reason, retrieve memory, and act in multi-agent environments.
4. **Training Efficiency:** Innovations like PipelineRL (in-flight updates) and Finite Dimensional Matching (FDM) are solving the computational bottlenecks that previously hindered the deployment of continuous-time deep learning models.

The "New Quant" is essentially a hybrid engineer: one who can derive a Fractional Riccati Equation, implement a Transformer with S5 layers, and train a Neural SDE using strictly proper scoring rules. The algorithms described in this report represent the current pinnacle of this convergence.

Works cited

1. (PDF) A Deep Reinforcement Learning Approach for Trading Optimization in the Forex Market with Multi-Agent Asynchronous Distribution - ResearchGate, accessed January 6, 2026, https://www.researchgate.net/publication/381006560_A_Deep_Reinforcement_Learning_Approach_for_Trading_Optimization_in_the_Forex_Market_with_Multi-Agent_Asynchronous_Distribution
2. PipelineRL: Faster On-policy Reinforcement Learning for Long Sequence Generation - arXiv, accessed January 6, 2026, https://arxiv.org/html/2509.19128v2
3. Can LLM-based Financial Investing Strategies Outperform the Market in Long Run? - arXiv, accessed January 6, 2026, https://arxiv.org/html/2505.07078v3
4. LOB-Bench: Benchmarking Generative AI for Finance – an Application to Limit Order Book Data - arXiv, accessed January 6, 2026, https://arxiv.org/html/2502.09172v1
5. [2410.03973] Efficient Training of Neural Stochastic Differential Equations by Matching Finite Dimensional Distributions - arXiv, accessed January 6, 2026, https://arxiv.org/abs/2410.03973
6. Efficient Training of Neural Stochastic Differential Equations by Matching Finite Dimensional Distributions | OpenReview, accessed January 6, 2026, https://openreview.net/forum?id=d4qMoUSMLT¬eId=LJD22sR0Oz
7. arxiv.org, accessed January 6, 2026, https://arxiv.org/html/2512.07162v1
8. Signature-Based Models in Finance | springerprofessional.de, accessed January 6, 2026, https://www.springerprofessional.de/en/signature-based-models-in-finance/51688532
9. QuantBench: benchmarking AI methods for quantitative investment from a full pipeline perspective | Request PDF - ResearchGate, accessed January 6, 2026, https://www.researchgate.net/publication/397348694_QuantBench_benchmarkin

g_AI_methods_for_quantitative_investment_from_a_full_pipeline_perspective